CS19541 - COMPUTER NETWORKS LAB MANUAL

EXPERIMENT 12

AIM: To implement an Echo Client-Server application using TCP and UDP sockets in C / Python/ Java.

DESCRIPTION:

- **TCP Socket Programming (Stream Sockets):** Utilizes Transmission Control Protocol to establish a reliable, connection-oriented byte stream using a three-way handshake. The echo server receives data sent by the client and sends back an exact copy.
- **UDP Socket Programming (Datagram Sockets):** Utilizes User Datagram Protocol for connectionless, unacknowledged datagram transmission. The client sends discrete messages directly to the server's IP and port, and the server echoes the message back.

FEATURES AND CAPABILITIES:

What we can do with TCP and UDP Sockets:

- Establish client-server socket connections across local or remote IP networks.
- Demonstrate byte-stream transmission and error recovery mechanisms using TCP socket calls.
- Demonstrate low-latency datagram transmission using UDP socket functions.
- Measure round-trip latency and payload integrity through echo back verification.
- Use cases are Echo protocols (RFC 862) are used for network diagnostics, testing protocol stacks, verifying latency, and checking host availability.

PROGRAM ARCHITECTURE OVERVIEW:

TCP Architecture:

- **Server Flow:** socket() -> bind() -> listen() -> accept() -> read/write loop -> close()
- **Client Flow:** socket() -> connect() -> write/read loop -> close()

UDP Architecture:

- **Server Flow:** socket() -> bind() -> recvfrom() -> sendto() -> close()
- **Client Flow:** socket() -> sendto() -> recvfrom() -> close()

STEP-BY-STEP PROCEDURE & IMPLEMENTATION CODE

Task 1: Implement TCP Echo Server & Client

1. Create the server socket bound to a designated port and start listening for incoming connections.
2. Accept connection requests from clients and echo back received data.

TCP Server Code:

Python

```python
import socket

def run_tcp_server(host='127.0.0.1', port=65432):
    server_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    server_socket.setsockopt(socket.SOL_SOCKET,
socket.SO_REUSEADDR, 1)
    server_socket.bind((host, port))
    server_socket.listen(1)
    print(f"[TCP SERVER] Listening on {host}:{port}...")

    conn, addr = server_socket.accept()
    print(f"[TCP SERVER] Connected by {addr}")

    try:
        while True:
            data = conn.recv(1024)
            if not data:
                break
            print(f"[TCP SERVER] Received: {data.decode('utf-8')}")
            conn.sendall(data)  # Echo back
            print(f"[TCP SERVER] Echoed back to client.")
    finally:
        conn.close()
        server_socket.close()
        print("[TCP SERVER] Connection closed.")

if __name__ == "__main__":
    run_tcp_server()
```

```
$ python tcp_echo_server.py
[TCP SERVER] Listening on 127.0.0.1:65432...
[TCP SERVER] Connected by ('127.0.0.1', 54210)
[TCP SERVER] Received: Hello, Computer Networks Lab - TCP Echo Test!
[TCP SERVER] Echoed back to client.
[TCP SERVER] Connection closed.
```

TCP Client Code:

Python

```python
import socket

def run_tcp_client(host='127.0.0.1', port=65432):
    client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    client_socket.connect((host, port))
    print(f"[TCP CLIENT] Connected to server {host}:{port}")

    message = "Hello, Computer Networks Lab - TCP Echo Test!"
    print(f"[TCP CLIENT] Sending: {message}")
    client_socket.sendall(message.encode('utf-8'))

    data = client_socket.recv(1024)
    print(f"[TCP CLIENT] Echo Received: {data.decode('utf-8')}")

    client_socket.close()

if __name__ == "__main__":
    run_tcp_client()
```

```
$ python tcp_echo_client.py
[TCP CLIENT] Connected to server 127.0.0.1:65432
[TCP CLIENT] Sending: Hello, Computer Networks Lab - TCP Echo Test!
[TCP CLIENT] Echo Received: Hello, Computer Networks Lab - TCP Echo Test!
```

Task 2: Implement UDP Echo Server & Client

1. Initialize a datagram socket on the server and bind it to a local port.
2. Receive packets from the client using recvfrom() and send the echoed payload back via sendto().

UDP Server Code Placeholder:

Python

```python
import socket

def run_udp_server(host='127.0.0.1', port=65433):
    server_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
    server_socket.bind((host, port))
    print(f"[UDP SERVER] Listening on {host}:{port}...")

    while True:
        data, addr = server_socket.recvfrom(1024)
        message = data.decode('utf-8')
        print(f"[UDP SERVER] Received from {addr}: {message}")
        server_socket.sendto(data, addr)  # Echo back
        print(f"[UDP SERVER] Echoed back to {addr}")
        break  # Exit after processing lab test frame

    server_socket.close()

if __name__ == "__main__":
    run_udp_server()
```

UDP Client Code Placeholder:

```python
import socket

def run_udp_client(host='127.0.0.1', port=65433):
    client_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)

    message = "Hello, Computer Networks Lab - UDP Echo Test!"
    print(f"[UDP CLIENT] Sending: {message}")
    client_socket.sendto(message.encode('utf-8'), (host, port))

    data, addr = client_socket.recvfrom(1024)
    print(f"[UDP CLIENT] Echo Received from {addr}:
{data.decode('utf-8')}")

    client_socket.close()

if __name__ == "__main__":
    run_udp_client()
```

TERMINAL EXECUTION OUTPUT

Task 3: Execution and Verification

1. Compile and run the TCP server, then launch the TCP client to send and verify echoed messages.
2. Compile and run the UDP server, then send datagram messages via the UDP client and confirm matching replies.

```
$ python udp_echo_server.py
[UDP SERVER] Listening on 127.0.0.1:65433...
[UDP SERVER] Received from ('127.0.0.1', 58912): Hello, Computer Networks Lab - UDP Echo Test!
[UDP SERVER] Echoed back to ('127.0.0.1', 58912)
```

```
$ python udp_echo_client.py
[UDP CLIENT] Sending: Hello, Computer Networks Lab - UDP Echo Test!
[UDP CLIENT] Echo Received from ('127.0.0.1', 65433): Hello, Computer Networks Lab - UDP Echo Test!
```

Result:

Thus the experiment to implement an Echo Client-Server application using TCP and UDP sockets has been studied and executed successfully.